

Daybook: an AI-powered daily companion

Architecture, tool choices and their reasoning, security design, and the problems solved along the way.

Live: daybookupload.vercel.app · Repo: github.com/SoulXD2006/Daybook · ~60 files · 17 routes · 6 database tables

01 The 30-second pitch

THE OPENER

Daybook is a full-stack personal productivity app built and deployed to production. Users type tasks in plain English — like “*remind me to call mom Thursday*” — and Google's Gemini LLM parses them into structured tasks with dates, priorities, and categories. It also generates a warm AI daily briefing, has a journal that surfaces patterns across entries over time, and tracks habit streaks with a calendar heatmap. It's a Next.js 16 app with TypeScript, Postgres via Prisma ORM, email/password auth with Auth.js, per-user data isolation, and it's live on Vercel with the database on Neon. Built end-to-end: schema design, API security, LLM integration with graceful fallbacks, responsive UI with dark mode, and the deploy pipeline.

02 Architecture

Daybook is a **monolithic full-stack Next.js app** — one codebase and one deployment serves the UI, the API, and the database access layer. No separate backend repo, no CORS, and shared TypeScript types between client and server.

```
Browser (React 19)
  → Next.js server (server components + API route handlers)
    → Prisma 7 (pg driver adapter) → Neon Postgres (serverless)
    → Google Gemini (gemini-2.5-flash, structured JSON output)
```

Request flow for a protected page like `/journal`:

- A **server component layout** checks the session (`auth()`) and redirects to `/signin` if absent — protection happens on the server before any UI ships.
- The page's **client component** renders, then fetches `/api/journal`.
- The **route handler** re-validates the session (never trust the client), scopes the query to `userId`, and returns JSON.

Note the **double check** — page-level redirect for UX, API-level 401 for security. The API check is the real boundary; the redirect is a courtesy.

03 Tech stack — and why each piece

TOOL	ROLE	WHY THIS ONE
Next.js 16	Full-stack framework	App Router: file-based routing, server components (auth checks without client JS), API routes in one project. Native Vercel deploys.

TOOL	ROLE	WHY THIS ONE
TypeScript	Language	End-to-end type safety: Prisma generates types from the DB schema, so a schema change breaks the build instead of production.
Tailwind v4	Styling	Design tokens defined once as CSS variables, consumed as utilities. Dark mode via a <code>.dark</code> class toggle.
Framer Motion	Animation	Declarative animations; <code>AnimatePresence</code> handles exit animations React can't do alone.
Prisma 7	ORM	Schema-first: one <code>.prisma</code> file generates SQL migrations and the typed client. v7 driver adapters talk to Postgres via pg.
Neon	Database	Serverless Postgres, free tier. Required because Vercel's filesystem is ephemeral — SQLite can't persist there.
Auth.js v5	Authentication	Credentials provider + JWT sessions in an encrypted <code>httpOnly</code> cookie; no session table needed.
bcryptjs	Password hashing	Adaptive salted hashing, cost factor 12. Passwords never stored in plain text.
zod	Input validation	Schema validation on every API write — never trust request bodies. Doubles as endpoint documentation.
Gemini	LLM	<code>gemini-2.5-flash</code> : fast, cheap, and supports structured output — hand it a JSON schema, get valid JSON back.
Vercel	Hosting	Zero-config Next.js deploys from GitHub; serverless functions, CDN, automatic HTTPS.

04 Data model

Six tables. **User** is the root — every other row belongs to exactly one user via a foreign key with `onDelete: Cascade` (delete an account and all its data goes with it).

```

User          id · email (unique) · name · passwordHash
■ ■ Task      title · category · priority · dueDate · completed · completedAt
■ ■ JournalEntry content · date              (one per user per day, upserted)
■ ■ Insight    content · createdAt            (cached AI observations)
■ ■ DailySummary content · date                (@@unique [userId, date] – cache key)
■ ■ Habit      name · emoji · archived
  ■ ■ HabitLog date · completed                (@@unique [habitId, date])

```

- **Composite unique constraints as cache keys.** `DailySummary's @@unique([userId, date])` enforces “one AI summary per user per day” in the database itself, and enables a single atomic upsert — no race condition.
- **HabitLog's @@unique([habitId, date])** makes double-logging a day impossible at the schema level.
- **Indexes follow the queries:** `@@index([userId, completed, dueDate])` on `Task` matches exactly how the `Today` view fetches.
- **Soft delete for habits** (archived flag) preserves streak history; **hard delete for tasks** where history has no value.

05 Feature deep-dives

Natural-language task parsing

User types free text → POST /api/tasks/parse → Gemini is called with a **JSON response schema** (title, dueDate, priority, category) plus the current date so relative phrases resolve → the structured result is validated and inserted.

```
// The critical Gemini config – forces valid JSON, no regex parsing
generationConfig: {
  responseType: "application/json",
  responseSchema: taskSchema, // typed Schema object
}
```

GRACEFUL DEGRADATION

If Gemini errors, times out, or no API key is configured, the endpoint catches it and stores the raw text as a plain task instead of failing. The user never sees an error; the app works without AI, just with less magic. The LLM is an enhancement, not a dependency.

AI daily summary

On loading the Today view, /api/summary gathers today's tasks, the week's deadlines, and unchecked habits, and asks Gemini for a two-to-three sentence briefing in a warm tone. The result is **cached per user per day** — the LLM runs once each morning, not on every page load. A deliberate API-cost decision.

Journal with memory

One entry per day (posting again the same day updates it — an upsert keyed on date range). The insight endpoint feeds the last 14 entries to Gemini asking for *one gentle recurring pattern*, only when 3+ entries exist, cached daily. Because entries persist in Postgres, the “memory” is real cross-session state, not a chat context window.

Habit streaks

Streaks are **computed on read, not stored** — a stored counter can drift; deriving from logs is always correct. The algorithm walks backwards from today:

```
function computeStreak(doneDays: Set<number>, today: Date): number {
  let cursor = startOfDay(today);
  if (!doneDays.has(cursor.getTime())) cursor = subDays(cursor, 1);
  // ^ today isn't checked yet – streak survives until midnight
  let streak = 0;
  while (doneDays.has(cursor.getTime())) { streak++; cursor = subDays(cursor, 1); }
  return streak;
}
```

The heatmap renders 90 days of logs as a CSS grid, GitHub-contribution style.

06 Auth & security

Authentication flow

- **Register:** POST /api/register → zod-validate → check email uniqueness → bcrypt.hash(password, 12) → create user → auto sign-in.
- **Sign in:** Auth.js credentials provider looks up the user, bcrypt.compare(), and on success issues a **JWT in an httpOnly cookie** (JavaScript can't read it — XSS can't steal it).

- **Every request after:** `auth()` decodes the cookie server-side; a helper `currentUserId()` is the first line of every API handler.

Defense layers

LAYER	IMPLEMENTATION
Session gate	Every API route returns 401 without a valid session; protected pages redirect via a server layout.
Data isolation	Every query filters on <code>userId</code> . Mutations use <code>updateMany/deleteMany({ id, userId })</code> — if the row isn't yours, count is 0 → 404. Prevents IDOR.
Input validation	zod schemas on every write: length caps, enum checks, email normalization.
Rate limiting	Sliding window: 5 registrations / 15 min per IP; per-user caps on the three Gemini endpoints.
Secrets hygiene	<code>.env</code> git-ignored; secrets only in local env + Vercel's dashboard; fresh <code>AUTH_SECRET</code> for production.
SQL injection	Non-issue by construction — Prisma parameterizes every query; no raw SQL anywhere.

VERIFIED BY TEST

Isolation was tested in both directions: a fresh account saw zero of the demo user's seven tasks, created its own, and the demo account couldn't see it either. Unauthenticated requests: 401. Wrong password: rejected.

07 Deployment pipeline

```
git push → Vercel build (install → generate → migrate → build) → production (serverless + CDN + HTTPS)

"build": "prisma generate && prisma migrate deploy && next build"
"postinstall": "prisma generate"
```

- **prisma generate** — the typed client is generated code, git-ignored, so it must be rebuilt on the build machine.
- **prisma migrate deploy** — applies pending SQL migrations to Neon during deploy. Schema changes ship themselves.
- Config lives in three environment variables in Vercel's dashboard: `DATABASE_URL`, `GEMINI_API_KEY`, `AUTH_SECRET` — none in code.

08 Problems hit — and how they were fixed

SQLite → Postgres migration for serverless

The app started on SQLite (zero-setup local file). Deploying to Vercel forced the realization that serverless filesystems are ephemeral — the SQLite file would vanish between invocations. Fixed by swapping the Prisma provider to `postgresql`, the driver adapter from `better-sqlite3` to `pg`, regenerating migrations, and moving to Neon. Because all DB access went through Prisma, **zero application code changed** — only config.

Adding a required foreign key to tables with existing rows

Adding `userId` (required) to tables that already had data made the migration fail — existing rows can't get a non-null column with no default. In dev: reset and reseed. In production the strategy would be three migrations: add nullable → backfill → make required.

Stale generated Prisma client

After schema changes, runtime errors like “`prisma.user` is undefined” appeared even though the schema was right. Root cause: the generated client is a build artifact that must be regenerated after every schema change. Automated via the `postinstall` hook.

TypeScript rejecting the Gemini response schema

The SDK's `responseSchema` failed type-checking because TypeScript widened enum literal types in the schema object. Fix: an explicit `Schema` type annotation on the constant — inference vs. annotation in practice.

Prisma 7 config not loading `.env`

Migrations failed with “connection url is empty” — Prisma 7's new `prisma.config.ts` doesn't auto-load `.env` the way earlier versions did. One-line fix: `import "dotenv/config"`.

Secrets nearly leaking via manual GitHub upload

GitHub's web “upload files” bypasses `.gitignore` — the mechanism keeping `.env` out of the repo. Caught it, verified the repo was private, switched to `git-push` deploys. Lesson: secrets hygiene is a pipeline property, not a one-time check.

09 Design Q&A

Why Next.js instead of separate React + Express?

One deployable, shared types, no CORS, server components allow auth checks before any JavaScript reaches the browser, and API routes give a real backend without a second repo. For a solo-built product, halving the surface area matters more than microservice flexibility that isn't needed yet.

Why JWT sessions instead of database sessions?

JWTs avoid a DB read on every request — the session verifies cryptographically from the cookie. Trade-off: no per-token revocation before expiry; rotating `AUTH_SECRET` invalidates all sessions as the blunt fallback. Acceptable here; a banking app would choose DB sessions. Knowing the trade-off is the answer.

How is cross-user data access prevented?

Server-verified `userId` from the session; every read filters on it; every mutation enforces ownership in the same atomic query (`updateMany({ id, userId })`) rather than check-then-act. IDs are CUIDs (non-sequential) so enumeration is impractical anyway — but the design doesn't rely on that.

What happens when the LLM fails or hallucinates?

Three guards: structured output constrains the response shape; `try/catch` fallbacks keep the app working (parse failure stores raw text, summary failure uses a template); `zod` validates before anything touches the DB. The AI can degrade the experience but can never corrupt data or take the app down.

How would this scale to 100k users?

The stateless app tier scales horizontally on Vercel already. Known bottlenecks: the in-memory rate limiter breaks across instances → Redis/Upstash; Postgres connections from many serverless functions → pooling (Neon's pooled endpoint); LLM cost → per-user daily caching already caps it at ~2 calls/user/day. Then add monitoring before guessing at further optimization.

How does dark mode avoid the flash of wrong theme?

Design tokens as CSS variables, redefined under a `.dark` class on `<html>`. A tiny inline script in `<head>` reads `localStorage` (falling back to the OS preference) and sets the class before first paint — that ordering is what prevents the flash.

10 Quick-fire glossary

TERM	ONE-BREATH DEFINITION
ORM	Maps database tables to typed code objects; Prisma generates the client from a schema file.
Migration	A versioned SQL script evolving the schema; committed to git, replayable anywhere.
JWT	Signed token carrying claims (user id); verified cryptographically, no DB lookup.
httpOnly cookie	Cookie invisible to JavaScript — session theft via XSS is blocked.
bcrypt	Deliberately slow, salted password hash; cost factor tunes slowness against hardware.
IDOR	Changing an ID in a URL to reach someone else's data; blocked by ownership-scoped queries.
Upsert	Update-or-insert in one atomic operation, keyed on a unique constraint.
Serverless	Functions spun up per request, no persistent machine — hence no local file storage.
Structured output	Constraining an LLM to return JSON matching a schema instead of free text.
Hydration	React attaching interactivity to server-rendered HTML in the browser.
Rate limiting	Capping requests per identity per time window.